

SNS-CA: ARITHMETIC CELLULAR AUTOMATON BASED ON STRUCTURAL NUMERICAL SYMMETRY

Structural Numerical Symmetry Cellular Automaton (SNS-CA) is a type of dynamical system based on the internal arithmetic properties of numbers, rather than spatial neighbor interactions.

AUTHOR:

Mikhail Yuryevich Yushchenko

Date: 09.08.2026

1. PARADIGM SHIFT:

Traditional cellular automata (such as Game of Life or Rule 30) are strictly bound to spatial interaction: a cell's state is dictated by its neighbors (the von Neumann-Ulam paradigm).

SNS-CA departs from this rule. In the proposed system, cells do not interact with each other. Instead, evolution is driven by the internal arithmetic symmetry of the number itself.

This establishes a new direction — **Arithmetic Cellular Automata**, where spatial interaction is replaced by number structure. The cell does not interact with neighbors; evolution is determined by internal structure. The rejection of the classical neighborhood paradigm defines SNS-CA as a unique type of dynamical system.

2. HOW IT WORKS:

The evolution of each cell is determined exclusively by the internal properties of its numerical state through three steps:

1. Splitting number N into m parts.
2. Multiplying each part by k .
3. Comparing the result with $N \cdot k$ by prefixes and suffixes.

Depending on the result, the cell transitions to one of five states:

- **F (Full)** — complete match.
- **B (Boundary)** — match at both start and end.
- **E (End)** — match only at end.

- S (Start)** — match only at start.
- N (No)** — no matches (cell transitions to zero state).

3. QUICK START:

3.1. Requirements

- Julia 1.6 or higher.
- FFmpeg (for GIF creation, optional).

3.2. Install Dependencies

using Pkg

```
Pkg.add(["Printf", "Random", "Images", "Plots", "ProgressMeter", "Dates", "DelimitedFiles"])
```

3.3. Run Simulation:

```
include("sns_ca.jl")
```

```
run_ca_sns_gif(100, 300, 2, 3, 10, 10^6, 5; seed=42)
```

4. PARAMETERS:

L: Grid size (L×L) [Default: 75]

steps: Number of iterations [Default: 300]

m: Number of parts for splitting [Default: 2]

k: Multiplier [Default: 3]

min_val: Minimum initial value [Default: 10⁴]

max_val: Maximum initial value [Default: 10⁸]

fps: GIF frame rate [Default: 5]

seed: Seed for reproducibility [Default: 42]

5. OUTPUT:

After execution, the following are generated:

Directory `ca_sns_frames/` — PNG frames of evolution.

GIF file — animation of evolution (e.g., `CA-SNS_m2_k3_2026-08-09.gif`).

CSV file — statistics by states (e.g., `stats_m2_k3_seed42.csv`).

6. ADDITIONAL FEATURES:

6.1. Analyze Single Trajectory

`analyze_single_trajectory(BigInt(123456), 2, 3, 20)`

6.2. Testing

`test_edge_cases()`

Verifies correctness on edge cases.

7. VISUALIZATION:

Color scheme for states:

-  Black — zero state (N).
-  Golden — full match (F).
-  Green — boundary match (B).
-  Red — end match (E).
-  Blue — start match (S).

8. LICENSE:

This project is distributed under the Creative Commons Attribution 4.0 International (CC BY 4.0) license.

9. CITATION:

When using this code in research, it is recommended to cite:

bibtex:

```
@misc{yushchenko2026sns,  
  author = {Yushchenko, Mikhail Yuryevich},  
  title = {SNS-CA: Arithmetic Cellular Automaton based on Structural Numerical Symmetry},  
  year = {2026},  
  publisher = {GitHub},  
  url = {https://github.com/Misha0966/New-project},  
  license = {CC BY 4.0}  
}
```

SOURCE CODE (sns_ca.jl)

[https://github.com/Misha0966/New-project/blob/main/Technologies/SOURCE CODE \(sns_ca \).jl](https://github.com/Misha0966/New-project/blob/main/Technologies/SOURCE CODE (sns_ca).jl)